

SAT-SA Lens — Project Report (SIH26157)

Sep 28, 2026 · @Manthan Chokshi

Project at a glance

SAT-SA Lens is an offline analytics tool that shows NCIIPC supervisors which critical-sector security teams need inspection, and proves why with evidence.

Field	Detail
Problem Statement ID	SIH26157
Title	Supervisory Analytics Tool for SOC Assessment (SAT-SA)
Organisation	National Technical Research Organisation (NTRO) / NCIIPC
Theme	Blockchain & Cybersecurity
Category	Software
Team	Domain Defenders
Product name	SAT-SA Lens
Idea submission deadline	30 September 2026

The tool ingests periodic SOC submissions (alerts, cases, escalations, asset lists) from many organisations, runs 14 explainable detectors, and ranks each organisation by supervisory risk. It runs fully air-gapped, with no cloud or external AI APIs.

The problem

NCIIPC reviews the security operations of India's critical-sector organisations by hand, and that review cannot scale to hundreds of organisations and millions of alerts.

Key terms

- **SOC (Security Operations Centre):** the team that handles security alerts. Each alert is investigated and either closed as a false alarm or escalated as a real incident.
- **Case management records:** the log of that work: who picked up each alert, what they wrote, how long it took, how it was closed.

- **CSE (Critical Sector Entity):** an organisation India cannot afford to have hacked, such as a power utility, bank, telecom operator or refinery.
- **NCIIPC:** the national body, under NTRO, that supervises the cyber resilience of CSEs.

Why manual review falls short. Supervisors sample a CSE's alert and case records and read them. These reviews regularly find weaknesses that policies, audits and KPI dashboards never show. Reading samples by hand is slow, inconsistent between examiners and covers only a small share of the data.

What NCIIPC asks for. A deployable tool that analyses SOC data from many CSEs at scale. It must flag entities needing attention, prioritise samples for manual review, detect operational weaknesses and explain every finding. It must support human judgement, not replace it.

Explicitly out of scope: acting as a SOC or SIEM, real-time monitoring, continuous log collection, or a national monitoring platform.

Gaps in the problem statement that we solve

The PS lists what to detect but leaves open how; these eight gaps are where SAT-SA Lens differs from a rules-plus-dashboard build.

Gap	Why it matters	How SAT-SA Lens solves it
No common data format	Every CSE uses different SOC tools, field names and severity scales	Canonical schema, per-CSE column mapping saved once, severity normalisation (P1, 5, Crit to Critical)
Execution gaps need the CSE's claims	The PS data list has records, not promises	A declared-commitments input (for example, escalate criticals in 15 min) checked against real case data
Negative space needs an expected baseline	You cannot spot what is missing without knowing what to expect	Expected picture from the asset list, peer groups and the entity's own history
No labelled data to prove accuracy	The PS asks for results comparable to expert review	Synthetic data with planted weaknesses and a ground-truth file; recall and precision per detector

Gap	Why it matters	How SAT-SA Lens solves it
Fast is not always lazy	Automated closures and good tuning look like bad behaviour	Automated closures separated; each finding carries an innocent explanation and a confidence score
Unfair peer comparison	A small co-operative bank is not SBI; tiny samples look extreme	Peer groups by sector and size, per-asset normalisation, small-sample smoothing
CSEs can game known rules	Closing at 6 minutes beats a 5-minute rule	Hard-to-fake signals (effort vs severity, note diversity, recurrence) plus random review sampling
Submissions are self-reported	Data can be incomplete or edited	Integrity checks on upload: ID gaps, time gaps, count mismatches, file hashes

Proposed solution: SAT-SA Lens

SAT-SA Lens is one offline web application with three modules, each finding tied to its reason and its evidence rows.

1. Execution Gap Detector. Flags work that looks done but was not: critical alerts closed in seconds, missing escalations, copy-paste investigation notes, and closures piled up just inside an SLA limit. It checks each CSE's declared commitments against its real case records.

2. Negative Space Finder. Flags evidence that should exist but does not: critical assets with no alerts, alert categories every peer has but this CSE lacks, sudden volume drops, and cases with no escalation trail.

3. Explainable Risk and Review Queue. Scores each CSE on 8 capability areas and ranks entities by risk. For each entity it picks a review sample (70% highest-risk records, 30% random) and exports a PDF evidence pack.

What the supervisor sees

1. Upload a CSE's submission and map its columns once.
2. Run the analysis; every run is logged with its data hash and rule versions.
3. Overview: all CSEs ranked by risk, with a capability heatmap.
4. Entity page: radar chart across 8 capability areas and the top 3 reasons in plain English.

5. Finding page: rationale, peer baseline, possible innocent explanation, confidence and the exact evidence rows.
6. Mark each finding Valid or Not valid; scores recalculate.
7. Export the review sample and the PDF evidence pack.

The 8 capability areas come from the PS: threat detection, investigation, escalation, incident response, security operations, governance and oversight, operational discipline, and cyber resilience.

Detector catalogue

SAT-SA Lens runs 14 analytic detectors and 5 data-quality checks; each returns a rationale, metrics, confidence, an innocent explanation and evidence row IDs.

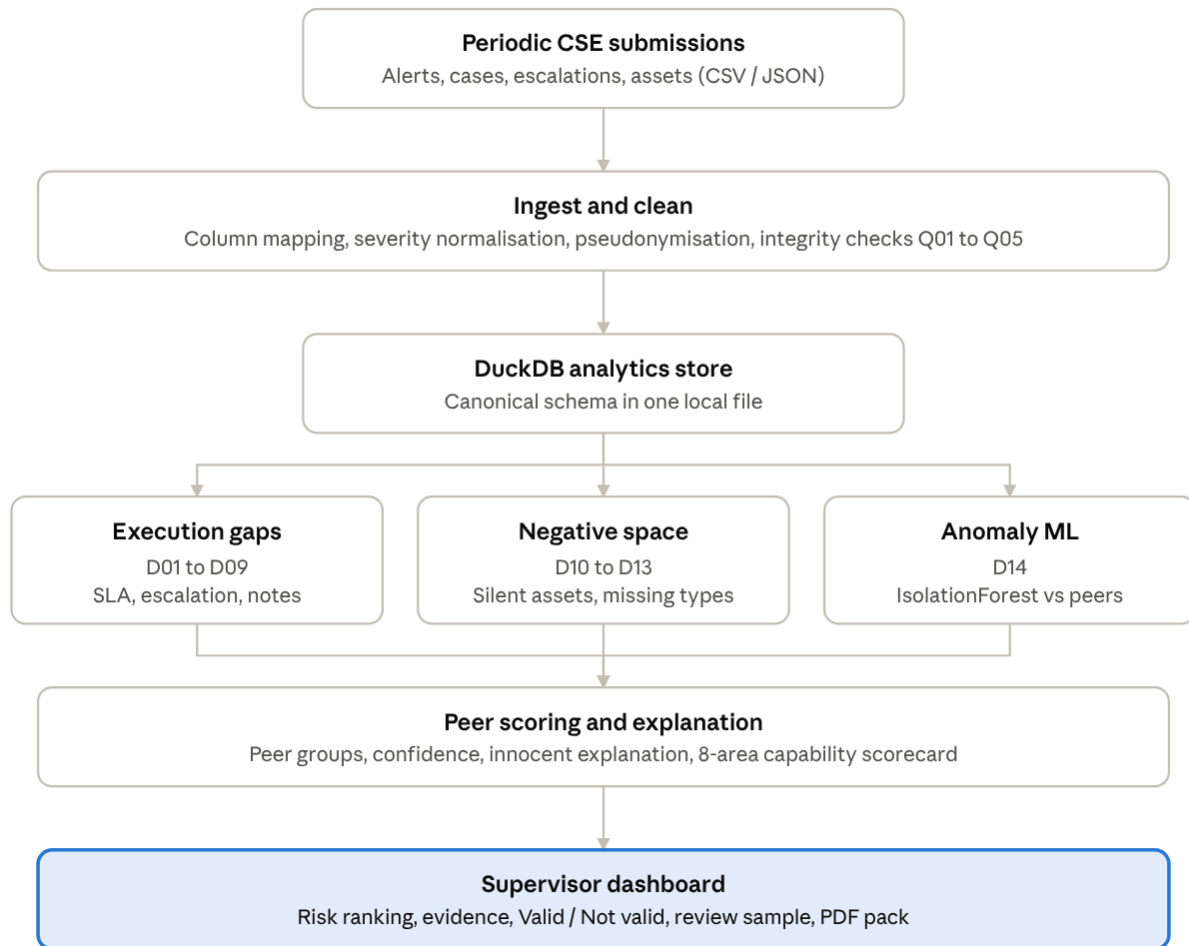
ID	Detector	Gap type	What it catches	Method	Capability area
D01	Fast critical closure	Execution gap	Critical or high alerts closed far faster than peers	Manual closures only, vs peer median	Investigation
D02	Critical without escalation	Execution gap	Critical alerts closed with no escalation record	Join alerts, cases, escalations	Escalation
D03	Commitment breach	Execution gap	Actual performance vs the CSE's declared SLAs	Declared threshold vs observed distribution	Governance and oversight
D04	Template notes	Execution gap	Copy-paste investigation notes	Local MiniLM embeddings, near-duplicate clusters	Investigation
D05	Shallow investigation	Execution gap	Notes with no IP, host, hash, user or action	Artefact extraction by pattern	Investigation
D06	Recurring, unremediated	Execution gap	Same rule on the same asset, closed again and again	Recurrence counts per rule and asset	Incident response
D07	Effort-severity mismatch	Execution gap	Investigation time that does not rise with severity	Time by severity, trend test	Operational discipline
D08	Threshold gaming	Execution gap	Closures piling up just inside an SLA limit	Distribution spike before the threshold	Operational discipline
D09	Implausible analyst workload	Execution gap	One analyst closing hundreds of alerts per shift	Closures per analyst per shift	Security operations
D10	Silent critical assets	Negative space	Critical assets with zero alerts	Asset list vs alert activity	Threat detection

ID	Detector	Gap type	What it catches	Method	Capability area
D11	Missing alert categories	Negative space	Categories at least 80% of peers have	Peer category coverage	Threat detection
D12	Volume drop	Negative space	Sudden fall vs the CSE's own history	Rolling baseline comparison	Cyber resilience
D13	Broken chain	Negative space	Cases with no alert, true positives with no escalation	Referential checks	Escalation
D14	Entity anomaly	Anomaly	Unusual CSEs not caught by rules	IsolationForest, top deviating features explained	Cyber resilience
Q01	ID gaps	Data quality	Missing alert ID ranges	Sequence check	Governance and oversight
Q02	Time gaps	Data quality	Periods with no records	Calendar coverage	Governance and oversight
Q03	Count mismatch	Data quality	Records differ from the declared total	Declared vs received	Governance and oversight
Q04	Timestamp regularity	Data quality	Timestamps that look generated	Interval regularity test	Governance and oversight
Q05	File hash	Data quality	Changes after submission	SHA-256 recorded per file	Governance and oversight

Technical approach

SAT-SA Lens is a single-machine Python and React application that runs with the network cable unplugged.

Every finding flows from raw submissions to an explained review queue



SAT-SA Lens architecture · 6 stages, 3 detector groups

Every stage runs locally; the run log records the data hash, code version and rule versions, so the same input always gives the same findings.

Layer	Technology
Frontend	React 18, Vite, TypeScript, Tailwind CSS, Recharts, TanStack Table; fonts bundled locally
Backend	Python 3.11, FastAPI, Uvicorn
Data engine	DuckDB (single local file), pandas for small transforms
Machine learning	scikit-learn IsolationForest; sentence-transformers all-MiniLM-L6-v2 bundled offline
Reports	reportlab PDF evidence packs

Layer	Technology
Security and trust	Pseudonymised analyst, user and IP fields; SHA-256 file hashes; role-based access; run audit log
Packaging	Docker Compose, exportable as an image bundle for air-gapped machines

Canonical data model. Seven tables: entities, declared commitments, assets, alerts, cases, escalations and submissions. Every detector returns the same finding object: gap type, capability area, severity, confidence, rationale, innocent explanation, metrics and evidence row IDs.

Scoring. Entities are compared only within their peer group (sector and size tier), with rates normalised per asset and per analyst. Small samples are shrunk toward the peer mean so a CSE is never flagged on a handful of alerts. Each detector maps to one capability area, giving a 0–100 score per area and an overall risk score with its top 3 reasons.

Offline guarantees. No calls to cloud AI APIs, CDNs or SaaS at runtime. Model updates arrive as signed offline bundles. The deployment is tested with Wi-Fi disabled.

Validation methodology

We prove the tool works by planting known weaknesses in realistic synthetic data and measuring how many it catches, mirroring what an expert manual review would find.

1. **Generate.** A seeded generator creates about 10 CSEs across power, banking, telecom and oil and gas, with 90 days of assets, alerts, cases and escalations.
2. **Plant.** Faulty CSEs receive 3 to 6 planted problems each, one injector per detector (for example, 78% of criticals closed in under 2 minutes, or a critical database server with no alerts). Three CSEs stay fully healthy.
3. **Plant look-alikes.** Two traps that must not be flagged: a CSE whose fast closures are all automated on info alerts, and a very small CSE with few alerts.
4. **Record ground truth.** Every planted problem is written to a ground-truth file with its expected detector and evidence IDs.
5. **Score.** The validation run compares findings to ground truth and reports recall, precision and false positives per detector, plus an overall "planted X, caught Y" figure.
6. **Check stability.** Re-running on the same data must give identical findings.

Target: overall recall of 85% or higher, with both look-alikes correctly left unflagged. Final numbers go here once the build completes. A pilot on real, anonymised NCIIPC data would replace synthetic ground truth with examiner findings.

Feasibility and risks

The project is feasible on a single machine with an open-source stack; the main risks are data realism and fairness, and each has a planned mitigation.

- **Technical:** proven open-source components (FastAPI, DuckDB, scikit-learn, React); every detector is unit-tested and validated against planted weaknesses.
- **Operational:** runs fully air-gapped on one machine; guided upload and column mapping handle each CSE's format.
- **Economic:** no licence fees; scales supervision without adding examiners.
- **Standards alignment:** NCIIPC CII guidelines, CERT-In Directions 2022, ISO/IEC 27001 and 27035, NIST CSF 2.0.

Risk	Mitigation
No real SOC data to build or test on	Synthetic data with planted weaknesses and ground truth; pilot on anonymised real data later
Every CSE uses different SOC tools and formats	Canonical schema plus saved column mapping per CSE
Fast closure may be legitimate automation	Automated closures separated; innocent explanation on every finding
Unfair comparison of small and large CSEs	Peer groups by sector and size; small-sample smoothing
CSEs learn the rules and game them	Hard-to-fake signals and random review sampling
Submitted data may be incomplete or edited	Integrity checks Q01 to Q05 and file hashes
Offline ML model needs updates	Signed offline model bundles, versioned in the run log

Impact and benefits

SAT-SA Lens lets NCIIPC analyse every submitted record instead of small manual samples, and points examiners to the gaps that matter.

Who	Benefit
NCIIPC supervisors	Full-data analysis, a prioritised review queue with evidence, consistent and auditable findings
Critical Sector Entities	Clear, evidence-backed feedback on SOC gaps; benchmarking against sector peers; blind spots fixed before attackers find them
Citizens	More reliable power, banking and telecom services; stronger national cyber resilience

User story. NCIIPC receives quarterly SOC data from 40 power utilities. SAT-SA Lens flags Utility-X: 78% of its critical alerts were closed in under 2 minutes, and its SCADA servers show no alerts at all. The supervisor reviews the 50 records the tool picked, confirms the gap, and sends the PDF evidence pack to the utility.

Wider effects

- **Social:** protects essential services and builds trust in critical-infrastructure oversight.
- **Economic:** scales supervision without adding examiners; one tool reusable across all critical sectors.
- **Security:** finds blind spots before attackers do; no data leaves NCIIPC.

Comparison with existing solutions

SIEM tools watch one organisation's security in real time and GRC tools check declared policies; neither audits how well many SOC's actually do their work.

Capability	SIEM tools	GRC / audit tools	SAT-SA Lens
Cross-CSE peer benchmarking	No	Partial	Yes
Execution-gap detection from case data	Some	No	Yes
Missing-evidence (negative space) detection	No	No	Yes
Evidence-linked, explained findings	Partial	Partial	Yes
Claimed SLA vs actual performance	No	Partial	Yes
Review-sample picker for examiners	No	No	Built-in
Runs fully air-gapped	Varies	Varies	Yes

This is a category-level comparison of typical tools, not of any single product.

Build plan and deliverables

The prototype is built in 13 phases; phases 1 to 6 already give an end-to-end demo, and later phases add depth.

1. Scaffold the repository: FastAPI backend, React frontend, health check.
2. Synthetic data generator with planted weaknesses and ground truth.
3. DuckDB database and ingest with pseudonymisation.
4. Detector framework, first detectors (D01, D02, D10, D11) and the validation harness.
5. Peer scoring, capability scorecard and API.
6. Dashboard: overview ranking and entity pages (first demo-ready version).
7. Remaining detectors D03 to D09 and D12 to D14.
8. Finding drill-down, negative-space view and supervisor feedback.
9. Upload of other formats: column mapping and integrity checks.
10. Review-sample picker, audit log and PDF evidence packs.
11. Validation page in the UI.
12. Offline Docker packaging and a Wi-Fi-off test.
13. Polish, README, architecture document and demo script.

Submission deliverables (as required by the PS)

- ☐ Source code link (GitHub)
- ☐ README with setup instructions

- ☐ Architecture document, 2 pages maximum
- ☐ Demo video, 2 minutes maximum
- ☐ Technical presentation, 5 slides maximum

References

Problem statement

- [SIH 2026 problem statements, official portal](#) (SIH26157)
- [SIH 2026 problem statement dataset with full descriptions](#)

Standards and law (to be verified and linked before submission)

- Information Technology Act, 2000, Section 70A (NCIIPC as nodal agency for CII protection)
- NCIIPC Guidelines for Protection of Critical Information Infrastructure
- CERT-In Directions under Section 70B(6) of the IT Act (April 2022)
- ISO/IEC 27001:2022 (information security management) and ISO/IEC 27035 (incident management)
- NIST Cybersecurity Framework 2.0 (2024)

Methods and technology

- Isolation Forest: Liu, Ting and Zhou (2008)
- MITRE ATT&CK tactic categories for alert classification
- DuckDB, scikit-learn, sentence-transformers (all-MiniLM-L6-v2), FastAPI and React documentation